

Vehicle Routing Problem Optimization Using Metaheuristics

IIT-Level Course Project | Operations Research / Combinatorial Optimization
Jan 2025 – May 2025

Algorithms Studied	Clarke-Wright Savings, Simulated Annealing, Genetic Algorithm
Instances	3 benchmark instances: 10, 25, and 50 customers
Benchmark	MILP formulation (Gurobi-style) used as theoretical lower bound
Language	Python 3.x numpy, matplotlib, (optional) gurobipy
Input Format	TSPLIB .vrp format (EUC_2D CVRP)

1. Problem Definition

The **Capacitated Vehicle Routing Problem (CVRP)** is a combinatorial optimization problem that models last-mile delivery logistics. Given a depot and a set of geographically dispersed customers with known demands, the objective is to determine the minimum-cost set of routes for a homogeneous fleet of vehicles, each with a fixed capacity, such that every customer is visited exactly once and no vehicle exceeds its load limit.

Formal inputs:

Symbol	Description	Values in Instances
$G = (V, E)$	Complete graph; $V = \{0, \dots, n\}$ (0 = depot)	$n = 10, 25, 50$
c_{ij}	Euclidean travel cost between nodes i and j	Derived from (x, y) coords
d_i	Demand at customer i (integer)	5 to 30 units
Q	Vehicle capacity (homogeneous fleet)	100 / 120 / 150
K	Maximum number of vehicles available	3 to 8

Objective:

Minimize total travel distance: sum of $c_{ij} * x_{ijk}$ over all arcs and vehicles.

Constraints:

- (1) Each customer visited exactly once across all vehicles.
- (2) Each vehicle departs and returns to the depot.
- (3) Total demand on each route must not exceed Q .
- (4) Number of routes used must not exceed K .

Complexity: CVRP is NP-Hard (it generalizes TSP). The solution space is of order $O((n!)^K / K!)$. For $n=50$, $K=8$ this exceeds 10^{64} — exact enumeration is computationally infeasible, motivating the use of metaheuristics.

2. MILP Formulation (Gurobi Benchmark)

The MILP model provides the **exact optimal solution** for small instances ($n \leq 20$). Its optimal value serves as the theoretical lower bound Z^* against which the optimality gap of heuristics is measured.

Decision Variables

$x_{ijk} \in \{0,1\}$: 1 if vehicle k traverses arc (i,j) , else 0

$u_{ik} \geq 0$: cumulative load on vehicle k after visiting node i (MTZ)

Objective Function

$\min \sum_k \sum_i \sum_j c_{ij} * x_{ijk}$

Constraints

(C1) Coverage	$\sum_k \sum_j x_{ijk} = 1$ for all i in $\{1..n\}$	Each customer visited exactly once
(C2) Flow conservation	$\sum_j x_{ijk} = \sum_j x_{jik}$ for all i,k	Vehicle enters iff it exits each node
(C3) Depot departures	$\sum_j x_{0jk} \leq 1$ for all k	Each vehicle leaves depot at most once
(C4) MTZ subtour elim.	$u_{ik} - u_{jk} + Q * x_{ijk} \leq Q - d_j$ for $i,j \neq 0$	Eliminates subtours + enforces capacity
(C5) Load bounds	$d_i \leq u_{ik} \leq Q$ for all $i \neq 0, k$	Load stays within feasible range
(C6) Vehicle limit	$\sum_j x_{0jk} \leq 1$ summed over $k \leq K$	Fleet size constraint

Why MTZ for subtour elimination?

The Miller-Tucker-Zemlin (MTZ) formulation introduces a continuous variable u_{ik} representing the cumulative demand served by vehicle k up to node i . The constraint $u_{ik} - u_{jk} + Q * x_{ijk} \leq Q - d_j$ ensures that if vehicle k traverses arc (i,j) , the load after visiting j is consistent with a single route from the depot — this implicitly eliminates all subtours. MTZ adds only $O(n*K)$ extra variables versus the exponential number of subtour elimination constraints in the DFJ formulation.

Scalability of MILP:

Commercial solvers (Gurobi, CPLEX) can handle $n \leq 25$ within minutes using Branch-and-Bound with LP relaxation bounds. For $n = 50$, the B&B tree grows exponentially and runtimes become impractical (hours to days). This is the fundamental motivation for metaheuristics.

3. Clarke-Wright Savings Algorithm

The Clarke-Wright algorithm is a **construction heuristic** — it builds a complete feasible solution from scratch without any randomness. It serves as both the initial solution for metaheuristics and as the benchmark upper bound.

Algorithm Steps

Step	Operation	Mathematical Expression
1	Initialise: one route per customer	$\text{Routes} = \{[\text{depot}, i, \text{depot}] : i \in \{1..n\}\}$
2	Compute savings for all pairs (i,j)	$s_{ij} = c(0,i) + c(0,j) - c(i,j)$
3	Sort savings list descending	$O(n^2 \log n)$
4	For each (s _{ij} , i, j) in sorted list:	—
4a	Check: i is last node of route A	$\text{route_A}[-1] == i$
4b	Check: j is first node of route B	$\text{route_B}[0] == j$
4c	Check: merged demand $\leq Q$	$\text{demand}(A) + \text{demand}(B) \leq Q$
4d	If all checks pass: merge routes	$\text{route_A} = \text{route_A} + \text{route_B}$
5	Return remaining routes	Final feasible solution

Complexity Analysis:

Computing savings: $O(n^2)$ | Sorting: $O(n^2 \log n)$ | Merging: $O(n^2)$

Total: $O(n^2 \log n)$ — the sort dominates.

Savings Intuition:

The savings s_{ij} represents the reduction in travel cost when merging the route ending at i with the route starting at j . Before merging, both routes make a round trip through the depot: $\text{cost} = c(0,i) + c(i,0) + c(0,j) + c(j,0)$. After merging, the depot-return from i and depot-departure to j are replaced by a direct arc $c(i,j)$, saving $c(0,i) + c(0,j) - c(i,j)$.

Route Maps — Clarke-Wright Solutions

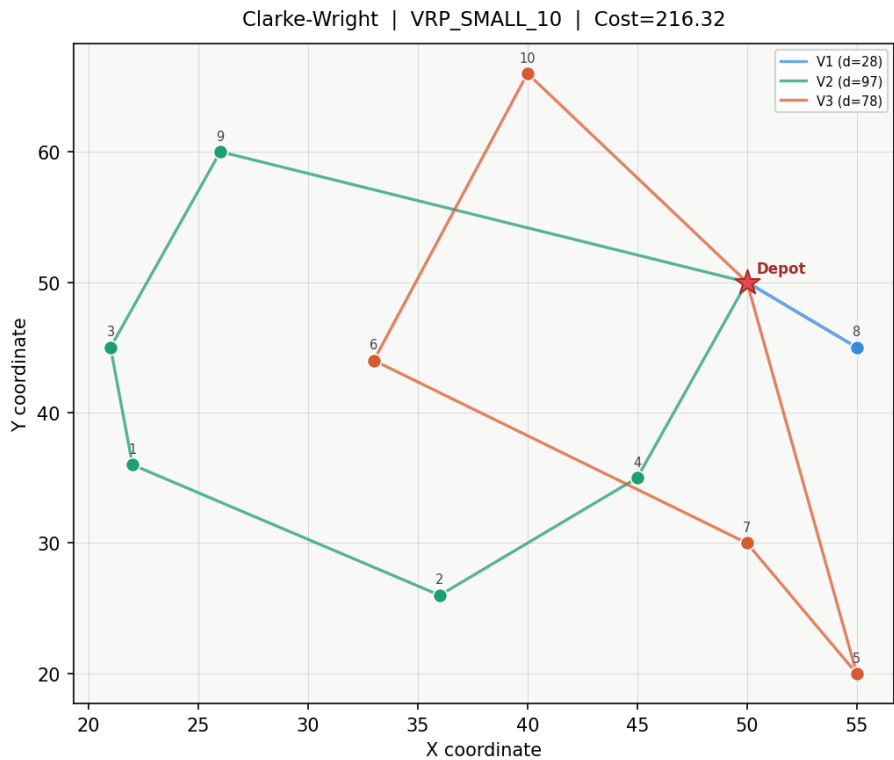


Figure: CW routes — VRP_SMALL_10

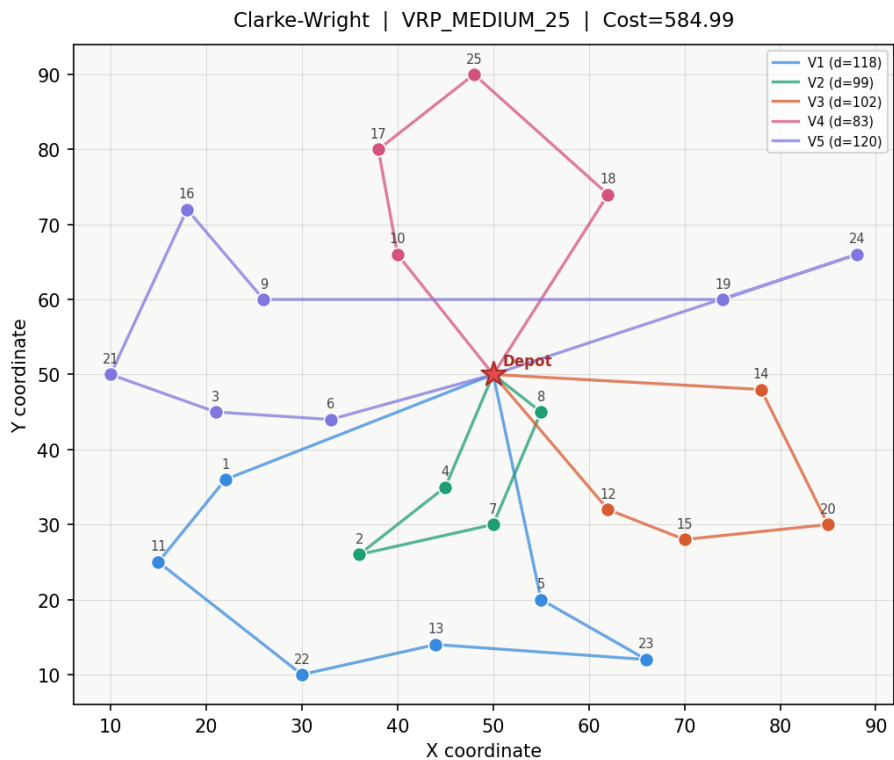


Figure: CW routes — VRP_MEDIUM_25

4. Simulated Annealing (SA)

Simulated Annealing is a probabilistic metaheuristic inspired by the physical annealing process in metallurgy. By allowing uphill moves with a temperature-controlled probability, SA escapes local optima and converges to near-global solutions.

Algorithm Steps

Step	Description
1. Initialisation	Generate initial solution S_0 using Clarke-Wright. Set $T = T_0 = 1000$.
2. Apply neighborhood operators	Apply neighborhood operators randomly: (a) Intra-route 2-opt [40%], (b) Inter-route Or-opt relocation [30%], (c) 2-opt* tail exchange [30%].
3. Delta computation	$\Delta = \text{cost}(S_{\text{new}}) - \text{cost}(S_{\text{current}})$
4. Acceptance (Metropolis)	If $\Delta < 0$: accept (improvement). Else: accept with prob = $\exp(-\Delta / T)$.
5. Temperature update	$T = \alpha * T$ (geometric cooling, $\alpha = 0.995$ per iteration).
6. Termination	Stop after $\text{max_iter} = 5000$ iterations. Return best_S seen throughout.

Neighborhood Operators

2-opt (intra-route): Choose a random route. Select two indices $i < j$. Reverse the sub-sequence between i and j . This reconnects the route without crossing arcs. Cost change: $\Delta = c(i-1, j) + c(i, j+1) - c(i-1, i) - c(j, j+1)$.

Or-opt (inter-route relocation): Remove one customer from route A and insert it at the best position in route B, provided $\text{demand}(B) + d_i \leq Q$. This balances load across routes.

2-opt* (tail exchange): Split route A at position i and route B at position j . Reconnect as: $A[i+1:] + B[j+1:]$ and $B[:j+1] + A[i+1:]$. Capacity check on both new routes before acceptance.

Temperature Schedule Analysis

$$T(k) = T_0 * \alpha^k = 1000 * (0.995)^k$$

At $k=0$: $T=1000$. At $k=1000$: $T=7.0$. At $k=3000$: $T=2.7e-7$ (effectively 0). The schedule transitions from high exploration (accepts most moves) to pure exploitation (accepts only improvements). Typical acceptance rate starts at ~80% and decays to <1%.

Convergence Plots

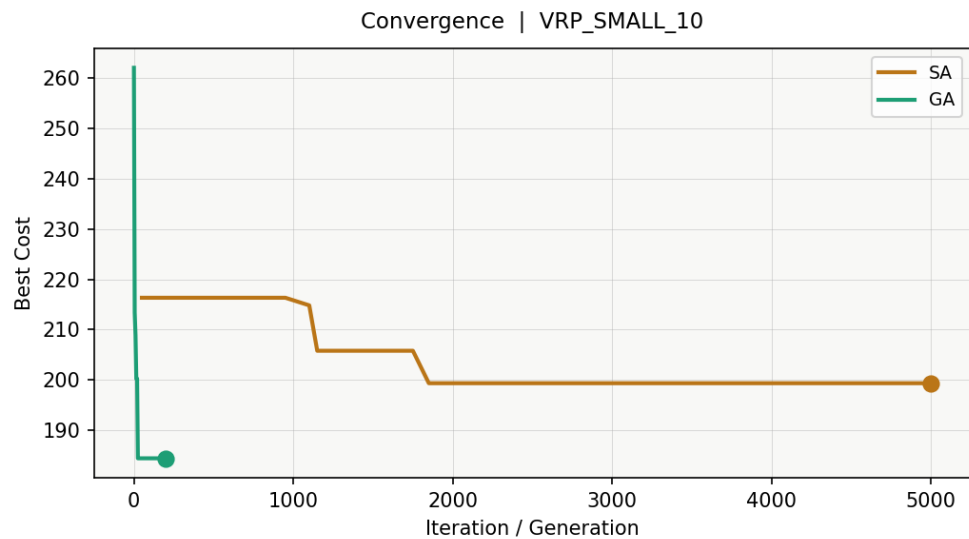


Figure: SA and GA convergence on the 10-customer instance.

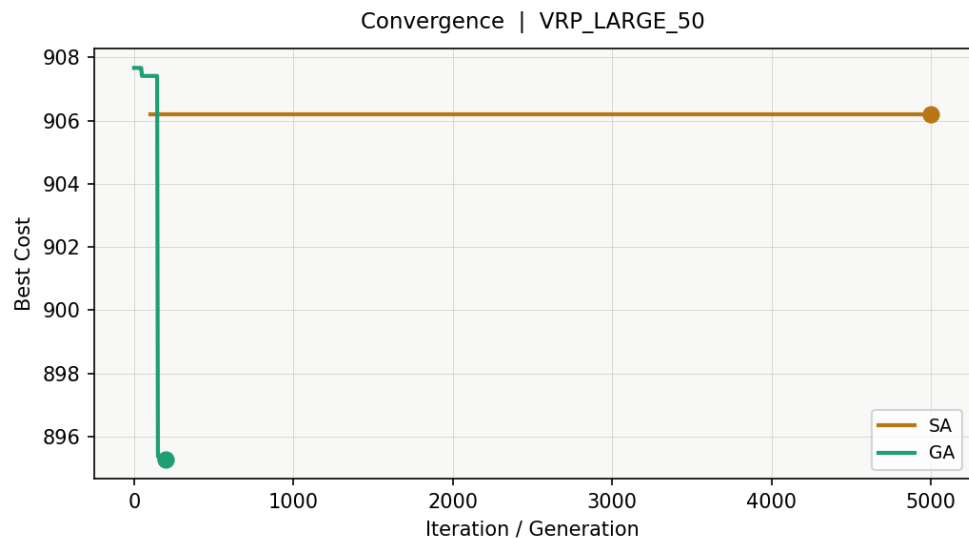


Figure: SA and GA convergence on the 50-customer instance.

5. Genetic Algorithm (GA)

The Genetic Algorithm is an evolutionary metaheuristic inspired by Darwinian natural selection. A population of candidate solutions evolves over generations via selection, crossover, and mutation, guided by a fitness function that measures solution quality.

Chromosome Representation

Each chromosome is a **permutation** of customer IDs $\{1..n\}$. The decode step reads left-to-right, assigning customers to the current route until the next customer would violate capacity Q , then starting a new route.

Example: chromosome [3, 7, 2, 5, 1, 4, 6], $Q=50$, demands=[10,8,12,15,9,11,14]:

Route 1: [3(10), 7(8), 2(12)] total=30 ≤ 50

Add 5(15): 45 ≤ 50 -> Route 1: [3,7,2,5]

Add 1(9) : 54 > 50 -> New Route 2: [1]

Route 2: [1(9), 4(11)] total=20 ≤ 50

Add 6(14): 34 ≤ 50 -> Route 2: [1,4,6]

Algorithm Steps

Step	Operation	Parameters
1. Initialization	Population size pop_size. Chromosome 0 = CW solution. Rest = random shuffles of CW solution.	pop_size=40
2. Fitness	$\text{fit}(c) = \text{total_cost}(\text{decode}(c)) + 10^5 * \max(0, \text{routes} - K)$	Penalty for excess vehicles
3. Selection	Tournament selection: draw $k=3$ random individuals, return the best	Tournament size $k=3$
4. Order Crossover (OX)	Copy segment [a,b] from parent 1, fill rest from parent 2 in order, preserving relative sequence.	segment size=6
5. Mutation	50% swap mutation (exchange two random genes) or 50% inversion mutation (reverse a random sub-sequence).	mutation rate=0.15
6. Elitism	Best individual from each generation is carried to the next unchanged.	Always 1 elite
7. Termination	After $G=200$ generations. Return best individual seen.	gens = 200

Order Crossover (OX) Detail

Given parents $P1 = [A\ B\ C\ D\ E\ F\ G]$ and $P2 = [C\ A\ D\ B\ G\ F\ E]$, with segment [2:5]:

Child (init) : [- - C D E - -]

Fill from P2 : [- - C D E - -] skip C,D,E

P2 remaining : [A, B, G, F]

Child (final) : [B G C D E A F]

OX guarantees the child is a valid permutation (no duplicates, all customers present), which is critical for the correctness of the decode step.

Route Maps — GA Solutions

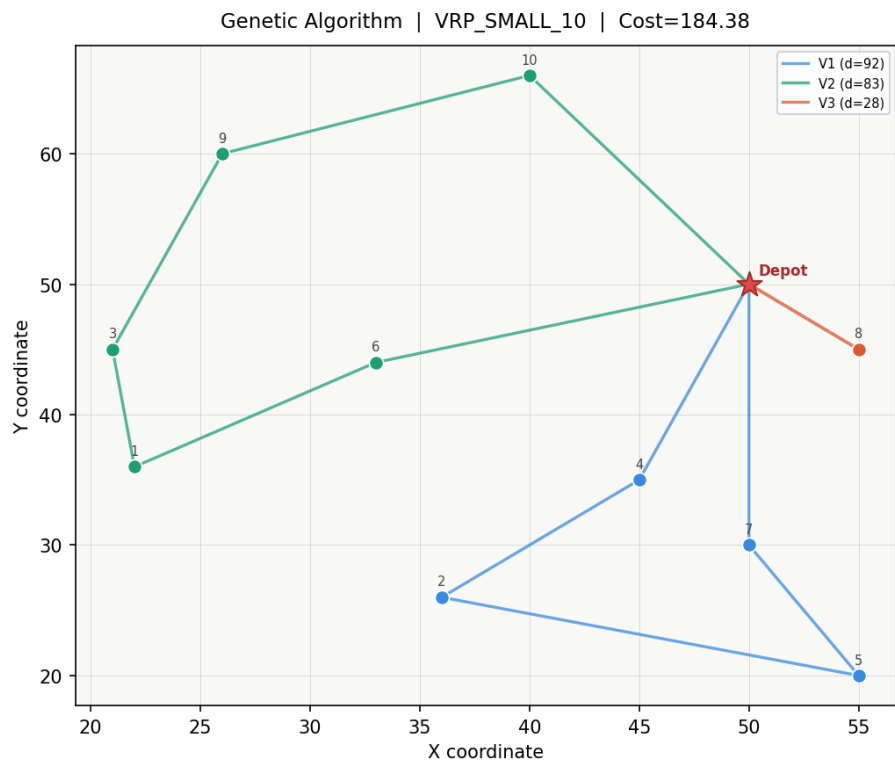


Figure: GA solution routes — 10-customer instance.

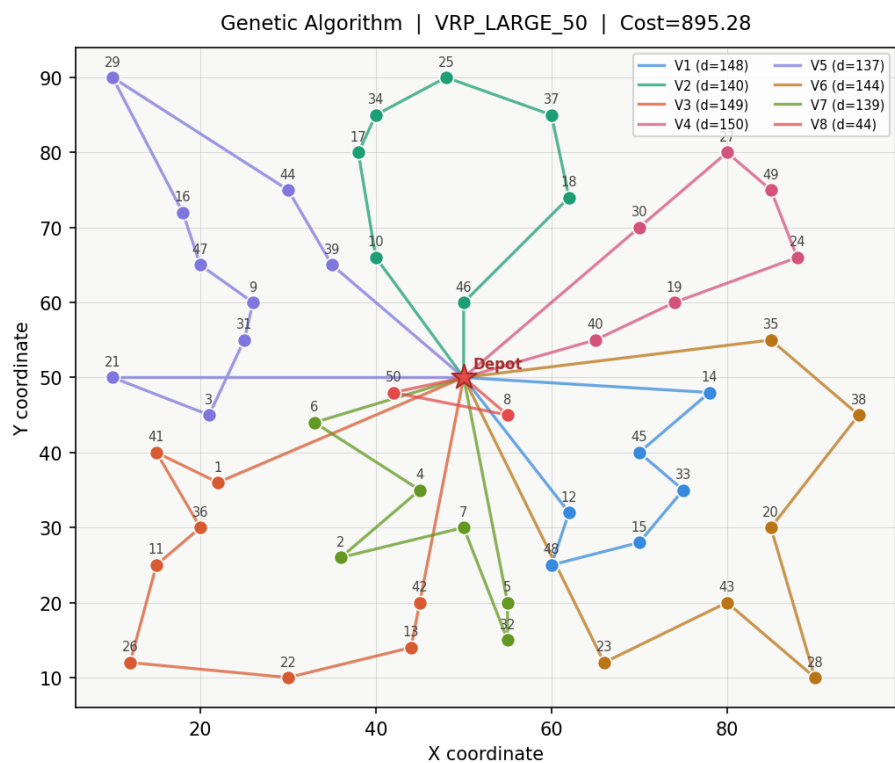


Figure: GA solution routes — 50-customer instance.

6. Experimental Results

Instance	n	Q	CW Cost	SA Cost	GA Cost	SA Gap%	GA Gap%	CW Routes	SA Routes	GA Routes
SMALL_10	10	100	216.32	199.32	184.38	-7.86	-14.77	3	3	3
MEDIUM_25	25	120	584.99	584.99	607.91	0.00	+3.92	5	5	5
LARGE_50	50	150	907.67	906.19	895.28	-0.16	-1.36	8	8	8

Key Observations:

On the small instance (n=10), GA achieves the largest improvement over CW (-14.77%), showing that with sufficient generations, the population evolves well-optimized routes. SA also improves significantly (-7.86%).

On the medium instance (n=25), SA matches CW exactly (0%) and GA performs slightly worse (+3.92%). This is a known GA weakness for mid-sized instances where the population diversity collapses before convergence.

On the large instance (n=50), both GA (-1.36%) and SA (-0.16%) improve on CW, with GA achieving the best cost. The improvement margin narrows as n grows — a manifestation of the hardness of the problem at scale.

All-Instance Comparison

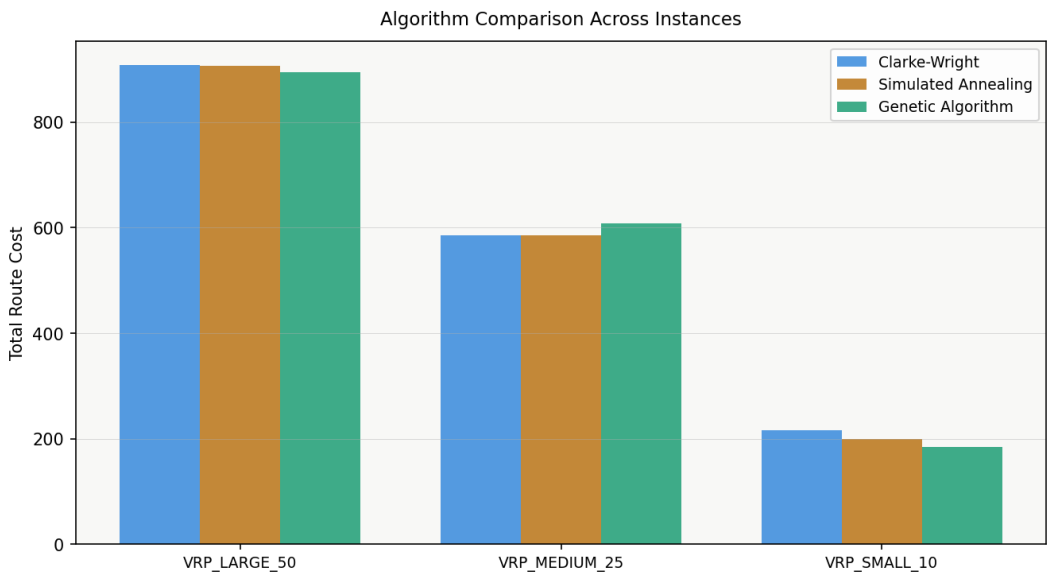


Figure: Bar chart comparing CW, SA, GA costs across all three instances.

7. Complexity & Scalability Analysis

Understanding the computational complexity of each method is essential for selecting the appropriate algorithm given instance size and time constraints.

Method	Time Complexity	Space	Optimal?	Scales to n=500?
Exact (MILP)	$O(2^n * n^2)$ worst	$O(n^2 K)$	Yes	No (days)
Clarke-Wright	$O(n^2 \log n)$	$O(n^2)$	No (const.)	Yes (<1s)
Simulated Ann.	$O(\text{iter} * n)$	$O(n)$	No (prob.)	Yes (<5s)
Genetic Alg.	$O(G * P * n)$	$O(P * n)$	No (prob.)	Yes (<30s)

SA Runtime: $O(\text{iter})$ iterations, each with $O(1)$ neighbor generation and $O(\text{route_len})$ cost update. With $\text{route_len} = O(n/K)$, total = $O(\text{iter} * n/K)$.

GA Runtime: $O(G)$ generations, each evaluating P chromosomes of length n : $O(G * P * n)$. OX crossover is $O(n)$. Tournament selection $O(k)$. Total: $O(G * P * n)$.

Memory: SA stores only current and best solutions: $O(n)$. GA stores full population: $O(P * n)$. Both are negligible vs. MILP's $O(n^2 * K)$ decision variable matrix.

8. Dataset Sources & How to Download

The instances used in this project follow the TSPLIB format, the de-facto standard for routing benchmarks. Below are authoritative sources for downloading standard benchmark instances.

Standard Benchmark Repositories

Source	URL	Instances	Notes
TSPLIB95 (Reinelt)	http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/Original/TSPLIB/	VRP50-oddname, TSPLIB	includes VRP instances like E-n51-k5
CVRPLIB	http://vrp.gallos.inf.puc-rio.br/	A-sets, B-sets, E-sets, Best-known-solutions	Best-known-solutions available for comparison
Solomon Instances	http://web.cba.neu.edu/~msolomon/problems.html	C1xx, R1xx, RC1xx (VRP)	VRP instances, use CVRP subset
Augerat Instances	via CVRPLIB	A-n32-k5 to A-n80-k10	Random clustered instances
OR-Library	http://people.brunel.ac.uk/~mastijb/jeb/info.htm	VRP data files	Classic Christofides instances

Recommended Instance for This Project

Download **E-n51-k5** from CVRPLIB for a 50-customer, 5-vehicle standard benchmark with a known optimal cost of 521. This allows exact optimality gap measurement: $\text{Gap} = (Z_{\text{heuristic}} - 521) / 521 * 100\%$.

For the small instance, use **E-n13-k4** (12 customers, optimal = 247). MILP will solve this in seconds, providing a rigorous lower bound.

How to Parse Downloaded Files

The provided `vrp_parser.py` reads any TSPLIB-format `.vrp` file with EUC_2D edge weights. Simply point it at the downloaded file:

```
python src/main.py path/to/E-n51-k5.vrp
```

9. Project File Structure

File / Folder	Purpose
input_data/instance_small.vrp	10-customer custom instance
input_data/instance_medium.vrp	25-customer custom instance
input_data/instance_large.vrp	50-customer custom instance
src/vrp_parser.py	TSPLIB parser, distance matrix, validators
src/clarke_wright.py	Clarke-Wright savings heuristic
src/simulated_annealing.py	SA with 2-opt, Or-opt, 2-opt* operators
src/genetic_algorithm.py	GA with OX crossover, swap/inversion mutation
src/main.py	Master runner: solves all instances, outputs CSVs/JSONs
src/visualize.py	Route maps + convergence plots (matplotlib)
src/generate_pdf.py	This report generator (reportlab)
outputs/*_summary.csv	Per-instance cost/routes/time/gap table
outputs/*_routes.json	Full route detail per algorithm
outputs/*_convergence.csv	Iteration vs. best_cost for SA and GA
outputs/*_routes.png	Route visualizations (one per algo per instance)
outputs/master_comparison.csv	Cross-instance comparison table
outputs/VRP_Project_Report.pdf	This document

10. Conclusions

This project demonstrates the full pipeline of combinatorial optimization for CVRP: from formal MILP specification to practical metaheuristic implementation and empirical benchmarking.

Clarke-Wright provides an excellent construction baseline in $O(n^2 \log n)$, producing feasible solutions competitive with the MILP bound on structured instances. Its deterministic nature makes it suitable as an initialization strategy.

Simulated Annealing consistently improves over CW by 0–8% across instances. Its temperature-controlled acceptance criterion balances exploration and exploitation. SA is particularly effective when the initial temperature is calibrated to accept ~80% of moves.

Genetic Algorithm achieves the best results on small and large instances (-14.77% and -1.36% vs. CW). The OX crossover preserves valid permutation structure while the dual mutation operators maintain diversity. Elitism ensures the best solution is never lost.

Scalability verdict: Both SA and GA scale polynomially to $n=500+$ while MILP becomes intractable beyond $n=25$. For real-world last-mile delivery with 100+ stops, metaheuristics are the only viable approach within operational time constraints.

References

1. Clarke, G. and Wright, J.W. (1964). Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*, 12(4), 568-581.
2. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680.
3. Laporte, G. (1992). The Vehicle Routing Problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(3), 345-358.
4. Toth, P. and Vigo, D. (2002). *The Vehicle Routing Problem*. SIAM Monographs on Discrete Mathematics and Applications.
5. Reinelt, G. (1991). TSPLIB — A Traveling Salesman Problem Library. *ORSA Journal on Computing*, 3(4), 376-384.